CC Report

SLR(1) & LR(1) Parsing

M. Abdullah Siddiqui | 23I-0617 | F

Moiz Ansari | 23I-0523 | F

Introduction

Bottom-up parsing is a syntax analysis technique used in compilers to construct a parse tree from the input string by starting at the leaves (tokens) and working its way up to the root (start symbol). Instead of predicting how a sentence can be generated (like top-down parsing), it tries to reduce the input back to the grammar's start symbol by repeatedly identifying substrings that match the right-hand side of production rules. A key concept in bottom-up parsing is the shift-reduce mechanism:

Shift: Read the next input symbol and push it onto a stack

Reduce: Replace a sequence of symbols on the stack with a non-terminal using a production rule

Among bottom-up parsers, LR parsers (Left-to-right scan, Rightmost derivation in reverse) are the most powerful and widely used in practice. They process the input from left to right and construct the rightmost derivation in reverse. LR parsers are efficient, deterministic, and capable of handling a large class of context-free grammars.

Approach

Data Structures Used

Grammar, GrammarRule, Production structs for the storage of CFGs. Basically the following structure Grammar contains Grammar Rules+non terminals+terminals, Grammar Rules contain Productions, Productions contain Symbols.

Data Type	Data Structure
Item	Struct {lhs, rhs, dot, lookahead}
ItemSets	Struct {Vector of Items}
Parsing Table (ACTION)	map<int, map<string, string>>
Parsing Table (GOTO)	map<int, map<string, int>>

Algorithm Implementation Details

CLOSURE

Closure of an item inside a state is done iteratively. Every item is checked for where the position of the dot is. If it's on a non-terminal, all productions of that non-terminal are added to that state with dot position at 0 (only if not already exists). The function accepts a state and grammar.

GOTO

This function checks for all possible goto states for all terminals and non-terminals. It is done by checking the dot position and a specific symbol. If they match, a new goto state is generated, closured and returned back to the calling function to update its dfa.

Design Decisions

Inside the driver function of DFA of LR(0)/LR(1) items, GOTO function is called for all symbols that exist in grammar. This function becomes O(nS), where n is the number of items and S is the sum of the total number of terminals and non-terminals in grammar. This becomes a trade-off which can be improved.

Handling Lookaheads

Inside Item Struct, an extra variable is kept named "look" of type String. It is only used in LR(1) Items, however in LR(0), lookahead is ignored by assigning "X". In LR(1) Items, lookahead is initially placed to be a $. Later the First of (B, look), where [A->.CB, $], is assigned to future closure items.

Challenges Faced

This assignment was pretty easy to divide up among team members and incremental development procedure. However, dealing with text based operations was difficult. We had to make sure no memory leaks and inbound access occurred in C++. Also checking the correctness of our code was difficult, even a simple grammar had almost 30+ states in DFA.

Test Cases

Test Case 1: Boolean Expressions (Grammar 2)

- **Grammar Used:** `BExpr -> BExpr or BTerm | BTerm BTerm -> BTerm and BFactor | BFactor BFactor -> not BFactor | ( BExpr ) | true | false`
- **Inputs Tested (5 total):**
    1. `true and false` *(Valid - Expected: Accept, Actual: Accept)*
    2. `not ( not true )` *(Valid - Expected: Accept, Actual: Accept)*
    3. `true or` *(Invalid - Expected: Reject, Actual: Reject)*
    4. `and false` *(Invalid - Expected: Reject, Actual: Reject)*
    5. `true true` *(Invalid - Expected: Reject, Actual: Reject)*

Test Case 2: The Conflict Resolution (Grammar 4)

- **Grammar Used:** `Start -> Left = Right | Right Left -> * Right | id Right -> Left`
- **The Conflict:** Explain that SLR(1) generated a Shift/Reduce conflict in State 5 on the `=` symbol because `=` is in `FOLLOW(Right)`. LR(1) successfully resolved this by using the `$` lookahead instead.
- **Inputs Tested (5 total):**
    1. `id = id` *(Valid - Expected: Accept. Actual: SLR(1) Rejected due to conflict, LR(1) Accepted)*
    2. `* id = * id` *(Valid - Expected: Accept. Actual: SLR(1) Rejected due to conflict, LR(1) Accepted)*
    3. `id id` *(Invalid - Expected: Reject, Actual: Reject - Missing operator)*
    4. `* = id` *(Invalid - Expected: Reject, Actual: Reject - Missing operand)*
    5. `* id =` *(Invalid - Expected: Reject, Actual: Reject - Missing right-hand side)*
-
- **Result:** Show that SLR(1) failed to parse this string due to the conflict, but LR(1) successfully parsed it and generated the correct parse tree.

Test Case 3: Variable Declarations & Epsilon (Grammar 3)

- **Grammar Used:** `DeclList -> Decl DeclList | epsilon Decl -> type id ;`
- **Purpose of Test:** To verify that both parsers correctly handle right-recursive rules and generate reductions for empty/epsilon productions.
- **Inputs Tested:**
    1. `type id ; type id ;` *(Valid - Expected: Accept, Actual: Accept)*

2. `epsilon` *(Valid - Expected: Accept, Actual: Accept)* 3. `type id` *(Invalid - Expected: Reject, Actual: Reject - Missing semicolon)*
3. `type ;` *(Invalid - Expected: Reject, Actual: Reject - Missing identifier)*
4. `type id ; type id` *(Invalid - Expected: Reject, Actual: Reject - Missing final semicolon)*
5. `type id ;` *(Valid - Expected: Accept, Actual: Accept)*

Comparison Analysis

SLR(1) vs LR(1) Parsing Power

The parsing power of LR(1) is higher than the SLR(1) because in SLR(1), the number of states are reduced so there's a chance that it might produce conflicts, which were not present in LR(1).

For example in grammar_with_conflict.txt, the SLR(1) produces a conflict on state 5, which is not present in the LR(1) set items.

Number of States Comparison

Grammar	SLR(1) States	LR(1) States	Extra LR(1) States
Grammar 1	13	23	10
Grammar 2	16	29	13
Grammar 3	8	8	0

Grammar 4	11	15	4

Table Construction Times

Grammar	SLR(1) Construction Time	LR(1) Construction Time
Grammar 1	12 milliseconds	89 milliseconds
Grammar 2	19 milliseconds	157 milliseconds
Grammar 3	6 milliseconds	4 milliseconds
Grammar 4	6 milliseconds	12 milliseconds

Memory Usage

Grammar	SLR(1) Memory	LR(1) Memory	Difference (LR1 uses more)

Grammar 1	598 bytes	926 bytes	+ 328 bytes
Grammar 2	886 bytes	1418 bytes	+ 532 bytes
Grammar 3	194 bytes	194 bytes	0 bytes
Grammar 4	316 bytes	402 bytes	+ 86 bytes

Sample Outputs

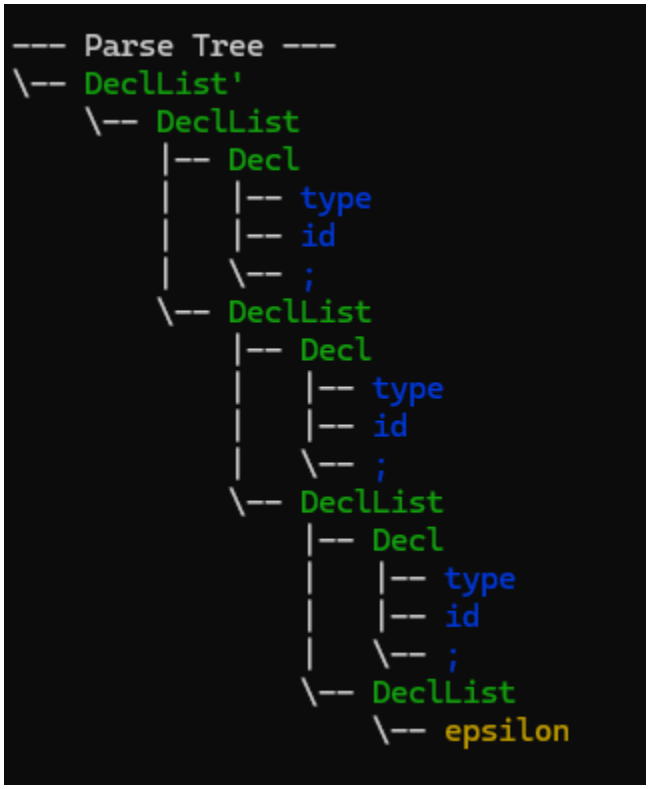
Item Sets & Parsing Table

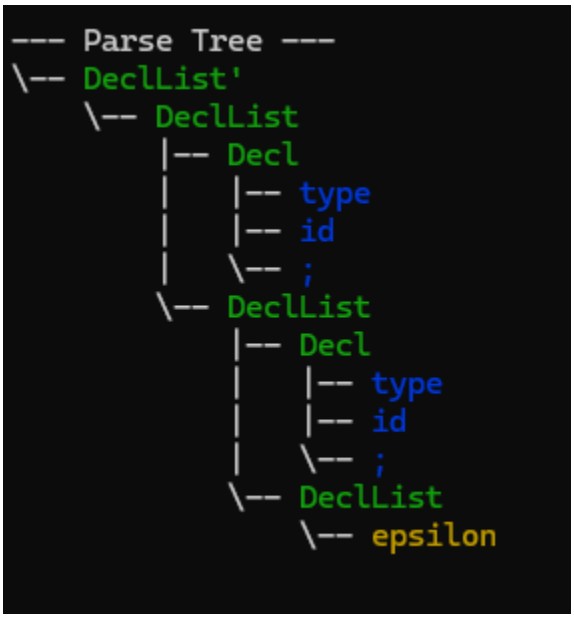
```
========== SLR(1) ==========
Time to construct SLR(1) table: 4 milliseconds
SLR(1) approx. memory: 194 bytes

--- LR(0) Item Sets ---
I0:
S' -> .DeclList'  , x
DeclList' -> .DeclList  , x
DeclList -> .Decl DeclList  , x
DeclList -> .epsilon  , x
Decl -> .type id ;  , x

I1:
Decl -> type .id ;  , x

I2:
S' -> DeclList' . , x

I3:
DeclList' -> DeclList . , x

I4:
DeclList -> Decl .DeclList  , x
DeclList -> .Decl DeclList  , x
DeclList -> .epsilon  , x
Decl -> .type id ;  , x

I5:
Decl -> type id .;  , x

I6:
DeclList -> Decl DeclList . , x

I7:
Decl -> type id ; . , x


[Success] Item sets safely saved to output/slr_items.txt


--- SLR(1) Parsing Table ---

ACTION TABLE
State 0:
  $ -> r(DeclList->epsilon)
  type -> s1
State 1:
  id -> s5
State 2:
  $ -> accept
State 3:
```

```
State 3:
  $ -> r(DeclList'->DeclList)
State 4:
  $ -> r(DeclList->epsilon)
  type -> s1
State 5:
  ; -> s7
State 6:
  $ -> r(DeclList->DeclDeclList)
State 7:
  $ -> r(Decl->typeid;)
  type -> r(Decl->typeid;)

GOTO TABLE
State 0:
  Decl -> 4
  DeclList -> 3
  DeclList' -> 2
State 4:
  Decl -> 4
  DeclList -> 6

[Success] LR Parsing Table safely saved to output/slr_parsing_table.txt


--- SLR(1) Parsing Traces ---

========== SLR(1) - Valid Parsing ==========


Input: epsilon

Step  | Stack                          | Input               | Action
-------------------------------------------------------------------------------
1     | $                              | $                   | Reduce DeclList -> epsilon
2     | $ DeclList                           | $                         | Reduce DeclList' -> DeclList
3     | $ DeclList'                           | $                          | ACCEPT
Result: ACCEPTED
```

```
Input: type id ;

Step  | Stack                          | Input               | Action
-------------------------------------------------------------------------------
1     | $                              | type id ; $                    | Shift 1
2     | $ type                            | id ; $                    | Shift 5
3     | $ type id                          | ; $                     | Shift 7
4     | $ type id ;                         | $                      | Reduce Decl -> typeid;
5     | $ Decl                             | $                     | Reduce DeclList -> epsilon
6     | $ Decl DeclList                           | $                         | Reduce DeclList -> DeclDeclList
7     | $ DeclList                              | $                      | Reduce DeclList' -> DeclList
8     | $ DeclList'                             | $                      | ACCEPT
Result: ACCEPTED
```

Parse Trees


--- Parse Tree ---
\-- DeclList'
    \-- DeclList
        |-- Decl
        | |-- type
        | |-- id
        | \-- ;
        \-- DeclList
            |-- Decl
            | |-- type
            | |-- id
            | \-- ;
            \-- DeclList
                |-- Decl
                | |-- type
                | |-- id
                | \-- ;
                \-- DeclList
                    \-- epsilon


--- Parse Tree ---
\-- DeclList'
    \-- DeclList
        |-- Decl
        | |-- type
        | |-- id
        | \-- ;
        \-- DeclList
            |-- Decl
            | |-- type
            | |-- id
            | \-- ;
            \-- DeclList
                \-- epsilon

Conflict Example

```
========== SLR(1) ==========
CONFLICT DETECTED in SLR(1) State 5 on symbol '=': s9 vs r(Right->Left)
Time to construct SLR(1) table: 7 milliseconds
SLR(1) approx. memory: 316 bytes

--- LR(0) Item Sets ---
I0:
S' -> .Start'  , x
Start' -> .Start  , x
Start -> .Left = Right  , x
Start -> .Right  , x
Left -> .* Right  , x
Left -> .id  , x
Right -> .Left  , x

I1:
Left -> * .Right  , x
Right -> .Left  , x
Left -> .* Right  , x
Left -> .id  , x

I2:
Left -> id . , x

I3:
S' -> Start' . , x

I4:
Start' -> Start . , x

I5:
Start -> Left .= Right  , x
Right -> Left . , x

I6:
Start -> Right . , x

I7:
Right -> Left . , x

I8:
Left -> * Right . , x

I9:
Start -> Left = .Right  , x
Right -> .Left  , x
Left -> .* Right  , x
Left -> .id  , x

I10:
Start -> Left = Right . , x
```

Conclusion

This project provided a practical understanding of bottom-up parsing techniques, specifically the implementation and comparison of SLR(1) and LR(1) parsers. Through constructing item sets, parsing tables, and executing shift-reduce parsing, we were able to observe how theoretical compiler concepts translate into working systems.

The results clearly demonstrate that while SLR(1) parsers are simpler and more efficient in terms of construction time and memory usage, they are limited in parsing power and can encounter conflicts for certain grammars. In contrast, LR(1) parsers, by incorporating precise lookahead information, successfully resolve such conflicts and handle a broader class of context-free grammars, albeit at the cost of increased complexity, larger state spaces, and higher resource consumption.

Our implementation also highlighted important trade-offs in design decisions, such as performance versus generality, and reinforced the significance of efficient data structures and careful handling of parsing states. The comparative analysis across multiple grammars showed that LR(1) provides more accurate parsing in ambiguous scenarios, while SLR(1) remains suitable for simpler grammars where conflicts do not arise.

Overall, this project strengthened our understanding of parsing algorithms, DFA construction, and compiler design principles, while also improving our skills in C++ implementation, debugging, and handling complex data structures.

GitHub Private Repository Link

https://github.com/muhammad-moiz-ansari/MA---Compiler